EVENTORA | PROJECT INTERVIEW GUIDE

Full-Stack Event Booking Platform

A beginner-friendly explanation of the project, its architecture, request flows, database design, security, debugging lessons, and interview questions.

> **One-line interview introduction**
> Eventora is a MERN-style event booking platform where users discover events, verify their identity through email OTP, submit booking requests, and manage bookings, while admins create events and approve or reject requests.

Prepared from the project source code. File paths in this guide are relative to the Eventora project root.

1. The Big Picture

Eventora has two applications that work together:

Part	Technology	Job
Frontend	React, Vite, React Router, Axios, Tailwind CSS, React Icons	Shows pages and collects user actions.
Backend	Node.js, Express	Provides API endpoints and business rules.
Database	MongoDB with Mongoose	Stores users, events, bookings, and OTP records.
Email	Nodemailer with Gmail SMTP	Sends account and booking OTP messages.
Authentication	bcrypt plus JWT	Protects passwords and API requests.

```
Browser (React)
   | Axios HTTP requests, JSON, Bearer token
   v
Express API (Node.js)
   | routes -> middleware -> controllers
   v
MongoDB (Mongoose models)       Gmail SMTP (Nodemailer)
```

Important distinction: React controls the interface, but the backend is the authority. The browser can display a dashboard button, but only backend middleware can decide whether the request is authenticated or admin-authorized.

2. Folder Structure

Path	Purpose
`client/src/main.jsx`	Frontend entry point. Mounts React and wraps it with AuthProvider.
`client/src/App.jsx`	React Router routes and common page shell.
`client/src/pages`	Home, login, registration, dashboards, event detail, payment pages.
`client/src/context/AuthContext.jsx`	Shared login state and authentication methods.
`client/src/utils/axios.js`	Axios instance and JWT request interceptor.
`server/index.js`	Express setup, middleware, route mounting, MongoDB connection, server startup.
`server/routes`	Maps HTTP methods and URLs to controllers.
`server/controllers`	Business logic for auth, events, and bookings.
`server/models`	Mongoose schemas for MongoDB documents.
`server/middleware/auth.js`	JWT verification and admin authorization.
`server/utils/email.js`	Nodemailer transporter and email functions.
`server/seed.js`	Development/demo data generator, not part of real registration.

3. How the Backend Starts

In `server/index.js`, the application:

1. Loads environment variables with `dotenv.config()`.
2. Creates an Express application.
3. Enables CORS so the Vite frontend can call the API.
4. Enables JSON request parsing with `express.json()`.
5. Mounts routes under `/api/auth`, `/api/events`, and `/api/bookings`.
6. Connects to MongoDB using `MONGO_URI` or `MONGODB_URL`.
7. Starts listening on port 5500 after the database connection succeeds.

```
const app = express();
app.use(cors());
app.use(express.json());
app.use('/api/auth', authRoutes);
app.use('/api/events', eventRoutes);
app.use('/api/bookings', bookingRoutes);
```

> **Interview phrase:** I keep routing, business logic, persistence, and authorization in separate layers. This makes each part easier to test and change.

4. Frontend Routing and Shared State

Routes are defined in `client/src/App.jsx`:

URL	Page
`/` and `/events`	Event listing and search interface
`/events/:id`	One event and booking flow
`/login`	Login and account OTP verification
`/register`	Registration and account OTP verification
`/dashboard`	Normal user bookings
`/admin`	Admin events and booking requests

`AuthProvider` wraps the app in `main.jsx`. Components call `useContext(AuthContext)` to access `user`, `login`, `register`, `verifyOTP`, and `logout`.

On startup, the provider reads `localStorage`. On login or verification, it saves the user and JWT. This lets a browser refresh keep the user logged in.

```
const { user, logout } = useContext(AuthContext);

// Axios adds the token to protected requests.
config.headers.Authorization = `Bearer ${token}`;
```

> **Security note:** localStorage is convenient, but JavaScript can read it. A production application may prefer secure, HttpOnly cookies with CSRF protection.

5. Registration and Account OTP Flow

```
Register.jsx
  -> POST /api/auth/register
  -> authController.registerUser
  -> bcrypt.hash(password)
  -> create User (isVerified: false)
  -> create OTP (account_verification)
  -> sendEmail()
  -> frontend shows OTP field
  -> POST /api/auth/verifyOTP
  -> compare OTP
  -> set isVerified: true
  -> create JWT
  -> save token and navigate
```

1. The user enters name, email, and password.
2. `AuthContext.register` sends JSON to the server.
3. The server rejects duplicate email addresses.
4. `bcrypt` hashes the password; the plain password is not stored.

5. The server creates an unverified user.
6. A six-digit OTP is stored with action `account_verification`.
7. Nodemailer sends the OTP through Gmail.
8. The frontend changes from the registration form to the OTP form.
9. The verify endpoint finds matching email, OTP, and action.
10. The account becomes verified, the OTP is deleted, and a JWT is returned.

6. Login Flow

1. The frontend sends email and password to `POST /api/auth/login`.
2. The backend finds the user by email.
3. `bcrypt.compare` compares the submitted password with the stored hash.
4. If the account is not verified, the backend creates and sends a fresh OTP.
5. The frontend shows the OTP input for that case.
6. Verified users receive a JWT containing their database ID and role.
7. Admins navigate to `/admin`; normal users navigate to `/dashboard`.

```
jwt.sign(
  { id: user._id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '5d' }
);
```

The JWT is not the user record itself. It is a signed proof containing identity claims. The middleware still loads the current user from MongoDB, which means a deleted user cannot continue using an old token.

7. OTP Generation and Expiration

```
Math.floor(100000 + Math.random() * 900000).toString()
```

This produces an integer from 100000 through 999999:

- `Math.random()` produces a decimal from 0 up to 1.
- Multiplying by 900000 creates a range below 900000.
- Adding 100000 makes it six digits.
- `Math.floor` removes decimals.
- `toString` converts the number to text.

In `server/models/OTP.js`, `createdAt` has `expires: 300`. Mongoose creates a MongoDB TTL index, so MongoDB removes the document approximately five minutes later. Successful verification also deletes it manually.

> **Production improvement:** use Node's `crypto.randomInt(100000, 1000000)` instead of `Math.random()` for security-sensitive OTP generation.

8. Event Browsing and Admin Event Creation

Users call `GET /api/events` from `Home.jsx`. The backend returns event documents, and React maps them into cards. Clicking View Details navigates to `/events/:id`.

Admin creation flow:

1. Admin opens the Create New Event form.
2. React sends title, description, date, location, category, seats, price, and image URL.
3. `protect` verifies the JWT.
4. `admin` checks `req.user.role === 'admin'`.
5. The controller maps frontend `category` to the existing schema field `cateogary`.
6. It maps `image` to `imageUrl` and initializes `availableSeats`.

> **Known code-quality issue:** the database field is spelled `cateogary`. It works because the code uses that spelling, but a cleanup migration should rename it consistently to `category`.

9. Booking Flow

```
EventDetail.jsx
  -> user clicks Confirm Registration
  -> POST /api/bookings/send-otp
  -> protect middleware loads req.user
  -> create event_booking OTP
  -> email OTP
  -> user enters OTP
  -> POST /api/bookings { eventId, otp }
  -> check OTP
  -> check event and duplicate booking
  -> create pending booking
  -> delete booking OTP
  -> admin sees request
```

A booking is not immediately confirmed. It is created with:

Field	Meaning
`status: pending`	Waiting for admin decision.
`paymentStatus: not_paid`	Payment has not been manually marked as paid.
`amount`	Copied from the event ticket price.

The project does not currently connect to a real payment gateway. Admin manually approves as paid or not paid.

10. Admin Booking Management

The admin dashboard calls `GET /api/bookings/my`. The backend now treats an admin differently:

- Normal user: query only bookings where `userId` is that user.
- Admin: query all bookings.
- Both event and user references are populated for display.

The dashboard calculates revenue and pending counts from the returned array. Admin can confirm a pending booking as `paid` or `not_paid`.

> **Interview honesty:** describe this as manual approval, not an automated payment system. The payment success/failure pages are currently static pages.

11. Database Models

User

`name`, unique `email`, hashed `password`, `role` (user/admin), and `isVerified`.

Event

Title, description, date, location, category, total seats, available seats, ticket price, image URL, creator reference, and timestamps.

Booking

References a user and event. Contains booking status, payment status, amount, and timestamps.

OTP

Email, OTP value, action type, and TTL-enabled creation time. The action separates account verification from booking verification.

```
User 1 ---- many Booking many ---- 1 Event
OTP is linked by email + action rather than a direct User ObjectId.
```

12. Middleware and API Security

`protect` performs these steps:

1. Reads the `Authorization: Bearer token` header.
2. Verifies the JWT signature with `JWT_SECRET`.
3. Loads the user by the token's ID.
4. Stores the user on `req.user`.
5. Rejects missing, invalid, or deleted-user tokens.

`admin` runs after `protect` and checks the role. Never rely only on hiding an admin button in React; the server must enforce authorization.

Password security

Passwords are hashed with bcrypt before storage. Hashing is one-way; login verifies a password against a hash rather than decrypting it.

Environment variables

Database URL, JWT secret, and Gmail credentials belong in `server/.env`. Never commit secrets. The README's old variable names should be updated to match the code's active names: `MONGODB_URL`, `EMAIL`, and `EMAIL_PASSWORD`.

13. Seed Data Versus Real Registration

`server/seed.js` is for development/demo data, not real registration.

It connects to MongoDB, clears events and bookings, preserves existing users, creates missing demo users, creates sample events, and creates sample bookings. It should not be used as part of the production startup or real signup flow.

> **Important:** running the seed script resets events and bookings. It preserves users in the current version, but real booking data should still not be reset in production.

14. Setup and Commands

Backend

```
cd server
npm install
npm run dev
```

`npm run dev` uses Nodemon. Production-style startup is `npm start`.

Frontend

```
cd client
npm install
npm run dev
```

Build and checks

```
cd client
npm run build
npm run lint
```

Demo data

```
cd server
node seed.js
```

The frontend Axios base URL is `http://localhost:5500/api`, while Vite normally runs on `http://localhost:5173`.

15. Known Limitations and How to Discuss Them

Issue	Good interview explanation
Seat accounting	Availability should be decremented atomically on confirmation using `availableSeats`, not by changing total capacity.
Admin rejection	Admin rejection should use an admin-specific authorization path instead of the user-owned cancellation rule.
Confirmation email recipient	The booking owner's email should be used, not the admin's email.
Search	The client sends a search term, but the backend must implement text filtering for it to work.
Booking date label	Use Mongoose's `createdAt`; the UI currently refers to a non-schema `bookedAt`.
Public registration role	Always force new public registrations to role `user`; never trust a client role field.
Testing	Add API tests for auth, OTP expiration, duplicate bookings, admin authorization, and seat concurrency.

16. Interview Introduction Script

“I built Eventora as a full-stack event booking platform. The React frontend uses React Router for navigation, Axios for API calls, and Context for shared authentication state. The Express backend exposes authentication, event, and booking APIs. MongoDB stores users, events, bookings, and time-limited OTP records through Mongoose. Passwords are hashed with bcrypt, while JWTs protect API requests. Registration and booking both use email OTP verification through Nodemailer. Admin users can create events and manually approve booking requests.”

17. Interview Questions and Answers

Q1. Explain the architecture of your project.

It is a MERN-style client-server application. React renders the UI, Express handles REST APIs, Mongoose maps MongoDB documents, and Nodemailer sends OTP emails. The frontend communicates with the backend over Axios JSON requests.

Q2. Why did you use React Context?

Authentication is needed by many unrelated components, such as Navbar, Login, dashboards, and EventDetail. Context avoids passing user and auth functions through every intermediate component.

Q3. How is a password stored?

The backend hashes it with bcrypt before saving. During login, bcrypt.compare checks the submitted password against the stored hash. The original password is never stored.

Q4. How does JWT authentication work here?

After successful login or verification, the server signs a token containing the user ID and role. Axios sends it as a Bearer token. The protect middleware verifies it, loads the user, and sets req.user.

Q5. How does the server know whether a user is an admin?

The JWT contains a role claim, but the middleware also loads the current database user. The admin middleware checks req.user.role === 'admin'. The server, not the UI, enforces access.

Q6. How is an OTP generated?

The current implementation uses Math.floor(100000 + Math.random() * 900000), which produces a six-digit number. For production, I would use crypto.randomInt because it is stronger for security-sensitive values.

Q7. How does OTP expiration work?

The OTP createdAt field has a MongoDB TTL of 300 seconds. MongoDB's background TTL process deletes expired records. Successful verification also deletes the record explicitly.

Q8. Why store the OTP in the database?

The API may run across requests or instances, so the verification endpoint needs shared state. MongoDB lets the server compare the submitted OTP and apply expiration consistently.

Q9. What is the booking lifecycle?

A user requests a booking after event OTP verification. The server creates a pending booking. An admin later confirms it as paid or not paid, or rejects it. The current system does not use a real payment gateway.

Q10. How do you prevent duplicate bookings?

Before creating a booking, the controller searches for an existing record with the same userId and eventId. A production version should also add a compound unique index to protect against race conditions.

Q11. What is the difference between totalSeats and availableSeats?

totalSeats is the original capacity. availableSeats is the remaining capacity. The current code has a seat-accounting bug in confirmation; the correct approach is to atomically decrement availableSeats and never modify totalSeats.

Q12. How would you handle two users booking the last seat at the same time?

Use an atomic MongoDB update with a condition such as availableSeats: { $gt: 0 } and $inc: { availableSeats: -1 }, then create the booking in a transaction or with carefully coordinated state.

Q13. What happens if the JWT belongs to a deleted user?

The middleware verifies the token, then tries User.findById. If no user exists, it returns 401 Not authorized, user not found. The frontend clears the stale session and redirects to login.

Q14. How would you improve error handling?

Add centralized Express error middleware that returns consistent JSON, avoid leaking stack traces, use meaningful status codes, and make the frontend read structured error messages.

Q15. How would you test this application?

I would test registration, wrong OTP, expired OTP, login, protected routes, role checks, event creation, duplicate bookings, last-seat concurrency, admin approval, and email-service failure. I would use API integration tests and frontend flow tests.

Q16. What would you change before production?

Use secure cookies or another safer token strategy, rotate exposed secrets, validate all request bodies, add rate limits for OTP requests, use crypto.randomInt, fix seat transactions, add tests, fix naming inconsistencies, and use a production email provider.

18. Final Revision Checklist

- Can I draw the browser -> API -> database flow?
- Can I explain protect and admin middleware?
- Can I explain bcrypt versus JWT?
- Can I explain both account OTP and booking OTP?
- Can I explain MongoDB TTL expiration?
- Can I explain pending, confirmed, and cancelled booking states?
- Can I name at least three current limitations and their fixes?
- Can I explain why seed.js is not part of real user registration?
- Can I describe how I would prevent overbooking?